セキュリティパッチ適用前後における アクセス制御の影響評価

埜口 裕矢[1,a)]　鐘本 楊[1,b)]　大庭 翔介[1,c)]　荒川 玲佳[1,d)]

概要：セキュリティパッチは脆弱性を修正する一方で，修正が不完全であったり，本来維持すべき機能性を損なったりする．特にアクセス制御脆弱性では，不要なアクセスが除去されたことに加え，正規のアクセスが維持され，意図しない権限付与や過剰な制限が生じていないことを確認する必要がある．しかし，Proof of Concept（PoC）コードや機能性テストによる評価では確認可能な実行経路が限定されるため，パッチによるアクセス制御の変化を網羅的に捉えることは難しい．そこで本研究では，セキュリティパッチ適用によるアクセス制御の変化に着目し，パッチ適用前のアクセス制御，パッチ適用によって期待されるアクセス制御，パッチ適用後のアクセス制御の三者を比較することで，脆弱性の修正が適切に行われたかを評価する手法を提案する．さらに，アクセス制御の脆弱性の修正が行われた実例を対象にケーススタディを行い，提案手法の有効性を示す．

キーワード：パッチ評価，アクセス制御脆弱性，不完全な修正

Impact Assesment of Access Control Before and After Security Patch Application

Yuya Noguchi[1,a)]　Yo Kanemoto[1,b)]　Shosuke Oba[1,c)]　Reika Arakawa[1,d)]

Abstract: Security patches fix vulnerabilities, but they may provide incomplete fixes or impair functionality that should otherwise be preserved. In particular, for access control vulnerabilities, it is necessary to verify not only that vulnerable access has been eliminated, but also that legitimate access is preserved and that no unintended privilege grants or excessive restrictions are introduced. However, evaluations based on Proof of Concept (PoC) exploits or functional tests are limited to the execution paths they cover, making it difficult to comprehensively capture changes in access control caused by a patch. Therefore, this study focuses on changes in access control introduced by security patches and proposes an automated method for evaluating whether an access control vulnerability has been appropriately fixed by comparing three access control states: the access control before patch application, the access control expected after applying the patch, and the access control after patch application. Furthermore, we conduct case studies on security patches for access control vulnerabilities to evaluate the effectiveness of the proposed method.

Keywords: Automatic Patch Evaluation, Access Control Vulnerabilities, Incomplete Fix

1. はじめに

ソフトウェアには，開発時には発見されなかった脆弱性が運用開始後に明らかになることがあり，これらの脆弱性を悪用した攻撃による被害を防ぐためには，適切なセキュリティパッチを自動で適用することが望ましい．一方で，

[1] NTT 社会情報研究所
NTT Social Informatics Laboratories
[a)] yuya.noguchi@ntt.com
[b)] yo.kanemoto@ntt.com
[c)] shosuke.oba@ntt.com
[d)] reika.arakawa@ntt.com

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

セキュリティパッチは脆弱性の原因となる処理を変更するものであり，本番環境等で適用する前に，既存の正常な機能を損なっていないか検証することが求められる．特に，今年 4 月から登場した Claude Mythos を始めとするフロンティア AI により発見される脆弱性の数が急増している [1]．同時に，適用すべきセキュリティパッチも同様に急増するため，システム運用者は今まで以上により瞬時にパッチ適用可否の判断が求められる [2]．しかし，パッチ適用はシステム障害にもつながるため，適用可否の判断は理想的にはシステム全体を含めた検証が必要であるが，検証には月単位の時間を要してしまう場合も存在する．さらに，パッチが脆弱性を部分的にしか修正していない場合や，修正に伴って本来許可されるべき処理まで制限する場合には，セキュリティ上または機能上の問題が残る可能性がある．したがって，セキュリティパッチ自動適用にあたっては，単にパッチを適用するだけでなく，その修正が意図した脆弱性を適切に除去し，かつ既存の正常な機能を損なっていないことを評価することが重要である．

セキュリティパッチの自動適用に関する先行研究では，脆弱性を再現する Proof of Concept（PoC）コードや機能性テストを用いて，パッチ適用後に脆弱な挙動が発生しなくなったことや，既存機能が維持されていることを確認する方法が主流である．これらの方法は，具体的な入力に対するプログラムの実行結果を確認できるため，既知の攻撃や想定された利用シナリオに対する修正の有効性を検証する上で有用である．しかし，テストではあらかじめ用意された入力や実行経路に対する挙動しか検証できないため，テストされていない実行経路に修正漏れが存在する可能性がある．

特に，アクセス制御に関する脆弱性は，本来許可されるべきでないアクセスを禁止することが脆弱性修正に必要であるが，単に脆弱なアクセスを拒否すれば十分であるとは限らない．修正によって，本来許可されるべきアクセスまで拒否され，既存機能の利用を妨げる過剰な制限となることや，別のアクセスが新たに許可されるようになることで，意図しない権限付与を導入する可能性があるためである．このような問題は，テストや PoC のみで十分に確認することが難しい [3]．

本研究では，大量の脆弱性に対するセキュリティパッチ検証を高速化することを目的とし，パッチによって生じるアクセス制御の変化に着目したパッチ適用後の機能性およびセキュリティを評価する手法を提案する．アクセス制御脆弱性は，プログラム固有の条件に依存して発生するため異なるプログラムに共通して適用可能な脆弱条件を定義することは難しい．一方で，アクセス制御脆弱性の修正を目的としたセキュリティパッチでは，パッチ適用によって，脆弱性に関するアクセスが除去されると同時に，正規のアクセスが維持されることが期待される．そこで本研究では，1. パッチ適用前のアクセス制御，2. パッチ適用によって期待されるアクセス制御，3. パッチ適用後のアクセス制御，の三者をモデル化し，比較する．これにより，パッチ適用前から期待されるアクセス制御への変更と，実際にパッチ適用後に生じた変更との差異を捉え，脆弱性修正が正しく実装されているかを評価する手法を提案する．

評価ではアクセス制御脆弱性の修正を実行したセキュリティパッチの実例を対象としたケーススタディによって，提案手法の有効性を示す．提案手法を実装するため，パッチ適用前後のアクセス制御の状態を把握するため Shi ら [4] の手法を活用した．パッチ適用によって期待されるアクセス制御の状態は，大規模言語モデル (Large Language Model: LLM) にパッチ適用前のアクセス制御情報，プログラムなどを入力として与えることで作成した．

本研究の貢献は以下の 2 点である．

- **セキュリティパッチによって生じるアクセス制御脆弱性に対し，パッチ適用前後および期待するのアクセス制御の三者をモデル化した概念を導入し，パッチ適用後に生じた変更との差異を捉える手法を提案した．**
- **2 件の実在する脆弱性のケーススタディを通じて，本手法の有効性を評価するとともに，手法の限界と今後の課題を明らかにした．**

本論文の構成は以下である．2 節では，関連研究を示す．3 節では，本研究の手法，実験対象について示す．4 節では，手法の実装方法について示す．5 節では，実験の評価結果を示す．6 節では，得られた結果に基づく考察を行う．7 節では，結論と今後の課題を述べる．

2. 関連研究

2.1 パッチの妥当性評価

パッチによって導入された変更が脆弱性の修正として妥当であるかを評価する研究が行われている．KLAUS [5] は Linux kernel の security patch を対象とし，パッチによって変更されるメモリアクセスを解析することで，不正確なパッチを検出する手法を提案している．Woo ら [6] は，National Vulnerability Database（NVD）から収集した 8,100 件の公開セキュリティパッチを対象として，パッチの有効性を大規模に分析した．また，セキュリティパッチに限らずパッチの妥当性を評価する取り組みも存在する．PatchGuru [7] は，pull request に含まれる自然言語情報から LLM を用いて実行可能な patch oracle を生成し，パッチ適用前後のプログラムの振る舞いが期待される変更を満たしているかを検証する．

これらの研究より，パッチを即時適用する前にパッチの妥当性評価を実施する必要性が示されている．本研究は，アクセス制御に特化してパッチの妥当性を評価する．

© 2026 Information Processing Society of Japan

2.2 アクセス制御脆弱性検知

アクセス制御脆弱性を検出するためには，アプリケーションに実装された認可処理を特定し，その意味をモデル化する必要がある．Sun ら [8] は，Web アプリケーションにおいてロールごとに暗黙に想定されるアクセス権限をソースコードから推定し，実際のアクセス制御との不整合から Forced Browsing の脆弱性を検出する手法を提案した．SeeAuthz [9] は，Java プログラムに実装された認可処理を静的解析し，実装によって強制されているアクセス制御を authorization graph として抽出する手法を提案している．一方，PeX [10] は Linux kernel を対象として，permission check とその後に実行される privileged operation との関係を定義し，それらを静的に解析することで認可チェックの欠落や不整合を検出するフレームワークを提案している．近年では，PCFinder [4] が Java Web アプリケーションにおける permission check を高精度に識別することで，アクセス制御脆弱性検出を支援している．また，Ote [11] は Web アプリケーションを対象に，アプリケーションの実行経路とデータベースアクセスとの関係を解析することで，実装からデータベースに関するアクセス制御を抽出する手法を提案している．

これらの研究は，実装に分散したアクセス制御処理を明示的なモデルへ変換できることを示しているが，脆弱性の判定にはドメイン知識や人間の確認が必要となるものが多い．

2.3 アクセス制御の変更・差分解析

アクセス制御の変更を意味的に解析する研究も行われている．Fisler ら [3] は，異なるアクセス制御ポリシー間の change-impact analysis を提案し，ポリシー変更によってアクセス判断がどのように変化するかを解析した．PDIFF [12] はアクセスログから時間的に変化するアクセス制御ポリシーを推定し，従来拒否されていたアクセスが新たに許可されるなどのポリシー変更を検出する．また，ACtest [13] はアクセス制御設定の変更前後に同一のアクセス要求を適用し，ALLOW から DENY，あるいは DENY から ALLOW へ変化する要求を抽出することで設定変更の影響を検査する．

これらの研究はアクセス制御の変更を検出可能にする一方，検出された変更がセキュリティパッチによって意図された修正内容と一致するかを判定することが難しく，最終的な判断は利用者に委ねているものが多い．

3. 提案手法

この節では，脆弱性修正によるアクセス制御の変化に着目し，パッチ適用前後のプログラムにおけるアクセス制御と，パッチによって期待されるアクセス制御を比較することで，脆弱性修正の妥当性を評価する手法を提案する (図 1)．

3.1 パッチ前後のアクセス制御

パッチによる影響を評価するため，本研究ではパッチ適用前後のプログラムについて，それぞれに実装されているアクセス制御を表現する．アクセス制御はプログラムが許可するアクセス，拒否するアクセスの両方を含む．

パッチ適用前のアクセス制御 (AC_{pre}) は，脆弱性を含むプログラムにおけるアクセスの集合である．アクセス制御脆弱性が存在する場合，AC_{pre} には，本来は禁止されるべきであるにもかかわらず許可されているアクセスが含まれる．

一方，パッチ適用後のアクセス制御 (AC_{post}) は，修正後のプログラムにおけるアクセスの集合である．パッチが適切であれば，脆弱性の原因となったアクセスは AC_{post} から除去され，さらに，脆弱性とは無関係な正規のアクセスは維持されることが期待される．

このため，AC_{pre} と AC_{post} の差分を求めることで，パッチによってどのアクセスが除去または追加されたかを表現できる．すなわち，$AC_{pre} \setminus AC_{post}$ をパッチによって失われたアクセス，$AC_{post} \setminus AC_{pre}$ をパッチによって新たに許可されたアクセスとする．これにより，パッチによる変更を単なるソースコード差分としてではなく，アクセス制御の変化として捉えることができる．

ただし，パッチ適用前後のアクセス制御を比較するだけでは，検出された変化が意図された修正であるのか，あるいは副作用であるのかを判断することは難しい．そこで本研究では，パッチによって期待されるアクセス制御を作成し，比較の基準として用いる．

3.2 パッチによって期待されるアクセス制御

パッチによって期待されるアクセス制御 (AC_{exp}) は，脆弱性を適切に修正したプログラムが満たすべきアクセス制御を表す．これは単にパッチ適用後のプログラムから取得されるアクセス制御ではなく，報告された脆弱性の内容や，修正の目的などから独立して導出される評価基準である．

アクセス制御脆弱性では，例えば「権限を持たない利用者による特定資源へのアクセスを禁止する」，「資源の所有者のみが更新操作を実行できる」といった，本来満たされるべきアクセス条件が存在する．本研究では，このような脆弱性修正によって満たされるべき条件を反映したアクセス制御を AC_{exp} とする．

AC_{exp} をパッチ適用後のアクセス制御とは独立して定義することにより，実際のパッチが修正者の意図した変更を実現しているかを評価できる．例えば，脆弱なアクセスが AC_{post} から除去されていたとしても，本来許可されるべきアクセスまで除去されている場合，AC_{post} と AC_{exp} の間には差異が生じる．同様に，パッチによって意図しな

© 2026 Information Processing Society of Japan

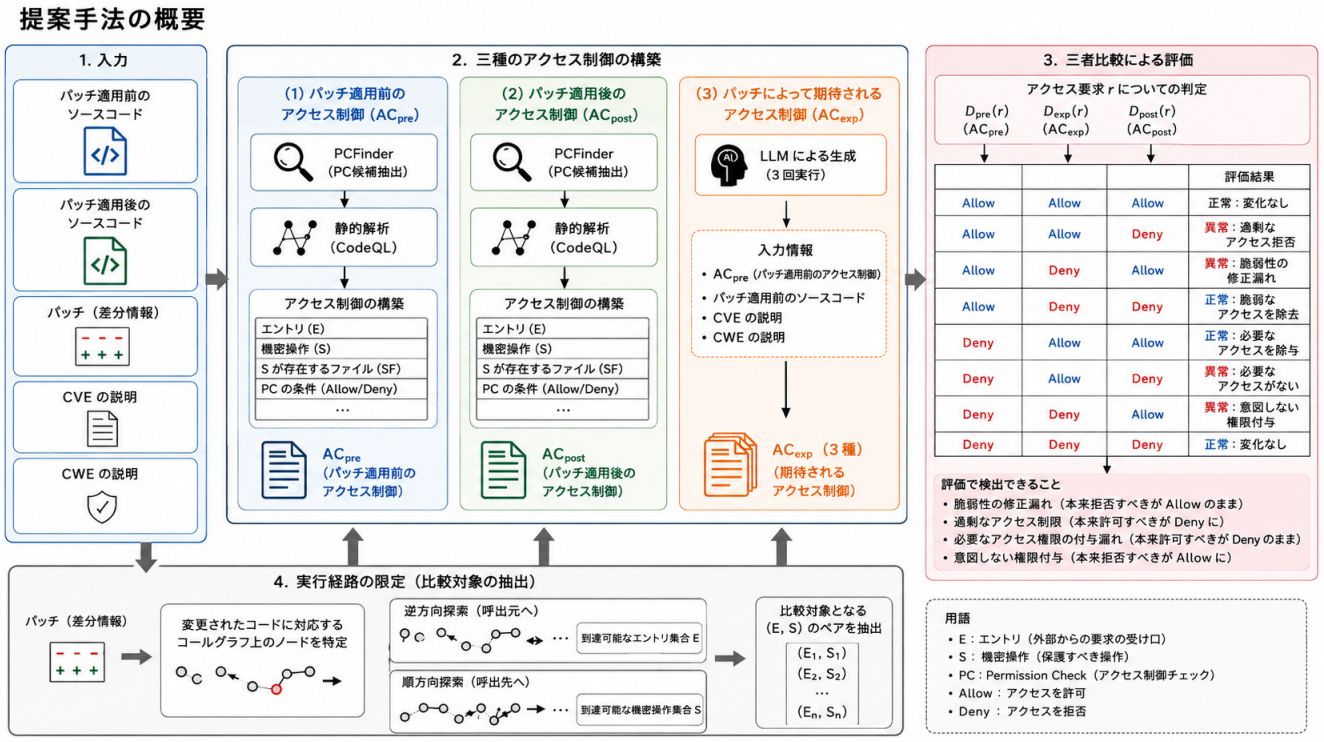


図 1　提案手法の概要

いアクセスが新たに許可された場合にも，そのアクセスは AC_{post} と AC_{exp} の差異として検出できる．

このように，期待されるアクセス制御を比較基準として導入することで，パッチ適用によってアクセス制御が「変化したか」だけでなく，その変化が「期待された方向への変化であるか」を評価可能とする．

3.3　三者比較による脆弱性修正の評価

本研究では，パッチ適用前のアクセス制御，パッチ適用後に期待されるアクセス制御，および実際のパッチ適用後のアクセス制御を比較することで，パッチによるアクセス制御の変更が適切であるかを評価する．

あるアクセス要求 (r) に対して，パッチ適用前のアクセス制御による判定を $(D_{pre}(r))$，パッチ適用後に期待される判定を $(D_{exp}(r))$，実際のパッチ適用後の判定を $(D_{post}(r))$ とする．各判定は，アクセスを許可する Allow，または拒否する Deny のいずれかを取る．本手法では，これら三つの判定の組合せに基づいて，パッチによる影響を分類する．

まず，$D_{pre}(r) = D_{exp}(r)$ である場合，そのアクセス要求はパッチによって変更されることが期待されていない．この場合，実際のパッチ適用後にも同じ判定が維持されていれば，パッチは当該アクセスに不要な影響を与えていないと判断する．一方，期待される判定とは異なる判定へ変化している場合，パッチによる意図しないアクセス制御の変更と判断する．

例えば，$(D_{pre}(r), D_{exp}(r), D_{post}(r))$ がそれぞれ (Allow,Allow,Allow) である場合，本来維持されるべきアクセスが適切に維持されているため正常と判断する．これに対して (Allow，Allow，Deny) となった場合，本来許可され続けるべきアクセスがパッチによって拒否されているため，過剰なアクセス制限と判断する．

次に，$D_{pre}(r) \neq D_{exp}(r)$ である場合，そのアクセス要求はパッチによって判定が変更されることが期待されている．例として，$D_{pre}(r)$ が Allow，$D_{exp}(r)$ が Deny である場合は，脆弱性の原因となるアクセスを禁止する変更に相当する．このとき，$D_{post}(r)$ が Deny であれば，期待されたアクセス制限が実現されており，脆弱性が適切に修正されたと判断する．一方，Allow のままであれば，期待されたアクセス制限が実現されていないため，脆弱性の修正漏れと判断する．

以上の分類も図 1 に示す．これにより，各アクセス要求について，パッチによって変更されるべきアクセスであるか，維持されるべきアクセスであるかを区別した上で，修正の成否や副作用を評価できる．具体的には，脆弱なアクセスの修正漏れ，正規アクセスに対する過剰な制限，必要なアクセス権限の付与漏れ，および意図しない権限付与を，同一の比較枠組みによって検出する．

4.　実装

この節では AC_{pre}，AC_{exp}，および AC_{post} の三者を構築し，比較するための実装について説明する．

© 2026 Information Processing Society of Japan

```
"allow": [
   {
     "entry": "AutoCompleteGal#handle",
     "condition": "accts.length != 0",
     "sensitive_operation": "Provisioning#
      getAccountById",
     "sensitive_file":
       "store/src/java/com/zimbra/cs/gal/
        GalSearchControl.java"
   ...
```

図 2 アクセス制御の例

4.1 パッチ前後のアクセス制御

AC_{pre}，AC_{post} を構築するため，Shi ら [4] の先行研究である PCFinder とそれを用いたアクセス制御抽出手法を用いた．

PCFinder はソースコード中でアクセス制御に関わる Permission Check (PC) 候補を抽出するツールである．PCFinder によって得られる情報は PC 候補であり，その PC がどのエントリから到達可能であるか，また，どの機密操作に対するアクセスを制御しているかまでは直接得られない．そこで，Shi らの手法と同様に，PCFinder によって抽出した PC と静的解析を組み合わせることで，アクセス制御を構築した．

具体的には，CodeQL [14] を用いて対象プログラムのコールグラフおよび制御フローグラフを構築する．まず，コールグラフを用いて，外部からの要求を受け付けるエントリから機密操作までの呼出し関係を特定する．さらに各メソッド内部の制御フローグラフを用いて，当該実行経路上に PCFinder によって抽出された PC が存在するか，PC は適切にアクセス制御を行っているかを調べる．実行経路上に複数 PC が存在している場合，それぞれの条件を AND で接続する．これにより，エントリから機密操作へ到達するまでに，どの PC が影響するかを求める．

アクセス制御は，エントリ (E)，機密操作 (S)，S が存在するファイル名 (SF)，両者の間に存在する PC の条件によって表現する．PC を満たした場合にへ到達可能な経路を Allow とし，PC によって実行が遮断される条件を Deny とする．また，同一の (E, S, SF) の組に対して複数の実行経路が存在する場合には，経路ごとにアクセス制御を構築する．

以上の処理をパッチ適用前およびパッチ適用後の双方のソースコードに対して実行し，それぞれのプログラムに実装されているアクセス制御を構築する．構築したアクセス制御の例を図 2 に示す．

4.2 パッチによって期待されるアクセス制御

パッチ適用によって期待されるアクセス制御，AC_{exp} は LLM を用いて生成する．LLM への入力として，パッチ適用前のアクセス制御である AC_{pre}，パッチ適用前のソースコード，CVE に記載された脆弱性の説明，および当該脆弱性に対応する CWE の説明を与える．

AC_{pre} は，脆弱性修正前にどのエントリからどの機密操作へのアクセスが可能であり，その経路にどのような PC が存在するかを LLM に与える情報である．さらに，パッチ適用前のソースコード，CVE および CWE の説明を与えることで，報告された脆弱性において問題となるアクセスと，そのアクセスを制限するために必要となる条件を推定させる．これらの情報に基づき，脆弱性が適切に修正された場合に満たされるべきアクセス制御を，AC_{exp} として生成する．

LLM の生成結果には，同一の入力を与えた場合であっても出力に揺らぎが生じる可能性がある．本研究では，上記の AC_{exp} 生成を同一条件下で 3 回実行し，各実行で得られたアクセス制御をそれぞれ評価対象とした．これにより，単一の生成結果のみに依存せず，LLM の出力のばらつきを考慮した評価を行う．

4.3 アクセス制御の比較対象

本研究では，パッチによる変更箇所と関係するコールグラフ上のノードを起点として，比較対象となる実行経路を限定する．パッチによる変更とは無関係なアクセス制御を省くためである．

まず，パッチによって変更されたコードに対応するコールグラフ上のノードを特定する．次に，当該ノードに対してコールグラフを逆方向に探索し，到達可能なエントリ E を求める．同様に，当該ノードからコールグラフを順方向に探索し，到達可能な機密操作 S を求める．それぞれに到達した (E, S) のペアは，コールグラフ上においてパッチで変更された箇所を経由した実行経路を持つ可能性があるため比較対象とする．

5. 評価

本研究では，提案手法がアクセス制御脆弱性に対するセキュリティパッチの妥当性を評価できるかを明らかにするため，以下の 2 つの Research Question（RQ）を設定する．

RQ1: 提案手法は，アクセス制御脆弱性を適切に修正したセキュリティパッチを正しく評価できるか．
RQ2: 提案手法は，不完全なセキュリティパッチに残存するアクセス制御脆弱性の修正漏れを検出できるか．

5.1 実験環境

実験は，macOS Sonoma 14.5 を搭載した計算機上で実施した．計算機の CPU には Apple M3 Pro を使用し，メモリ容量は 36 GB である．また，本手法において LLM を用いる処理には Claude Opus 5 を使用し，LLM を利用した解析および各種処理の実行環境として Claude Code を

© 2026 Information Processing Society of Japan

用いる．

5.2 実験対象

本実験では，アクセス制御に関する脆弱性のうち，CWE-862（Missing Authorization）のような，本来必要とされる認可処理が適切に実施されないことにより，権限を持たない主体による操作が可能となる脆弱性を対象とする．すなわち，必要な認可処理の欠落や，既存の認可処理を迂回する実行経路によって，本来許可されるべきでない操作が実行可能となる脆弱性を対象とする．よって，実験対象として，CVE-2020-10194 [15] および CVE-2025-27622 [16] を修正するセキュリティパッチを用いる．

CVE-2020-10194: 企業向けメール・グループウェアである Zimbra Collaboration Suite のメールボックスサーバに存在した認可処理の欠落に関する脆弱性である．Zimbra では，組織内のユーザやメールアドレスを検索するために Global Address List（GAL）を利用する．脆弱な実装では，認証済みユーザが GAL 検索を行う際，要求で指定された GAL 同期用アカウントが自身と同一ドメインに属するかを確認していなかった．このため，本来アクセスを許可されていない他ドメインの GAL 同期用アカウントを指定して処理を実行することが可能であった．修正パッチでは，要求元ユーザと指定された GAL 同期用アカウントのドメインを比較し，異なる場合にはアクセスを拒否する認可処理が追加されている．CVE-2020-10194 はこのパッチによって修正され，その後，同一の認可処理に関する修正漏れは報告されていない．したがって，本実験ではパッチ適用後に実装されたアクセス制御が期待されるアクセス制御と一致するため $AC_{exp} = AC_{post}$ となることを期待し，RQ1 の評価に利用する．

CVE-2025-27622: ソフトウェアのビルドやテスト，デプロイなどを自動化する CI/CD サーバである Jenkins に存在した脆弱性である．Jenkins では，ビルド処理を実行する計算機を Agent として管理しており，各 Agent の設定は `config.xml` として取得できる．Agent の設定には暗号化された秘密情報が含まれる場合があるため，Agent の設定を変更できる Agent/Configure 権限を持たないユーザに対しては，これらの秘密情報を表示しないことが期待される．しかし，脆弱な Jenkins では，Agent/Extended Read 権限のみを持つユーザが REST API または CLI を介して Agent の `config.xml` を取得した場合にも，暗号化された秘密情報がマスクされずに出力されていた．修正パッチでは，Agent/Configure 権限の有無を確認し，その権限を持たない場合には秘密情報をマスクしてから `config.xml` を返す処理が追加された．しかし，CVE-2025-27622 に対する修正では，REST API および CLI から config.xml を直接取得する経路は保護されたものの，Agent をコピーして新しい Agent を作成する別の HTTP 経路に同等の認可処理が適用されていなかった．そのため，Agent/Configure 権限を持たないユーザであっても，Agent のコピー機能を経由することで，秘密情報を含む設定を取得できる状態が残存した．この不完全な修正は，後に CVE-2025-31721 [17] [18] として報告され，再度修正された．したがって，CVE-2025-27622 は，パッチによって追加されたアクセス制御だけでは期待されるアクセス制御を完全には満たさない $AC_{post} \setminus AC_{exp} = \overline{\phi}$ の事例として，RQ2 の評価に利用する．

表 1 CVE-2020-10194 に対するアクセス制御抽出結果

	総アクセス制御数	パッチ関連数
AC_{pre}	38,492	58
AC_{exp}(平均)	38,492.0	58.0
AC_{post}	38,496	58

表 2 CVE-2020-10194 におけるパッチ関連のアクセス制御の比較結果

AC_{pre}	AC_{exp}	AC_{post}	件数
Allow	Allow	Allow	43.7
Allow	Allow	Deny	1.0
Allow	Deny	Allow	12.3
Allow	Deny	Deny	1.0

それぞれの脆弱性について，パッチ適用前のプログラムとパッチ適用後のプログラムを解析し，実装されているアクセス制御を抽出する．さらに，CVE および CWE の説明とパッチ適用前のプログラムをもとに，パッチによって期待されるアクセス制御を生成し，これら三者の比較を行う．なお，実験では PCFinder ツールが解析可能な Java 言語で実装されたプログラムのみを対象とするが，実装上の制約であり手法自体の制約ではない．

5.3 評価結果

5.3.1 CVE-2020-10194 に対する結果

CVE-2020-10194 に対して， AC_{pre} および AC_{post} と AC_{exp} との比較を行った．

表 1 が CVE-2020-10194 からアクセス制御を抽出した結果，表 2 がパッチに関連するアクセス制御の結果である．今回のアクセス制御抽出手法では，ある実行経路における Allow 条件の否定が Deny の条件であるので，図 1 のパッチ適用前の箇所を Allow として固定して，表 2 でアクセス制御の変化を評価した．

$(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Deny, Deny) となる，すなわち正しく脆弱性を修正すると期待されるアクセス制御は AC_{exp} を 3 回生成したすべてで 1 件であった．これは，パッチによって追加された if 文の条件式が，同様に AC_{exp} でも推測されていたためである．

一方，$(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Deny, Allow) となる，脆弱性の修正漏れが期待されるアクセス制御が 12.3 件あった．AC_{exp} を LLM が生成する際に CVE の説明文から過剰に防御が必要として，追加したためである．

© 2026 Information Processing Society of Japan

表 3　CVE-2025-27622 に対するアクセス制御抽出結果

	総アクセス制御数	パッチ関連数
AC_{pre}	4,880	409
AC_{exp} (平均)	4,884.3	413.3
AC_{post}	4,925	454

表 4　CVE-2025-27622 におけるパッチ関連のアクセス制御の比較結果 (平均値)

AC_{pre}	AC_{exp}	AC_{post}	件数
Allow	Allow	Allow	313.3
Allow	Allow	Deny	100.3
Allow	Deny	Allow	0.0
Allow	Deny	Deny	7.0

また，AC_{exp} の試行別で評価結果の偏りが大きく，3 回中 1 回の試行で 36 件現れたため，大きな値となっていた．

また，$(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Allow, Deny) となるものは 1 件存在したが，if 条件式の等号，不等号の関係が入れ替わっただけであったため，変更がされていないものと同等であると考えられる．

以上より，RQ1 の回答は以下となる．

RQ1 に対する回答： アクセス制御脆弱性を適切に修正したセキュリティパッチに対して，提案手法は修正された脆弱性を正しく反映した．しかし，AC_{exp} の生成に LLM を利用していることが原因で，修正漏れが存在すると誤検知されるケースがあるため，現状の手法では必ずしも正しく評価されるとは限らない．

5.3.2　CVE-2025-27622 に対する結果

CVE-2025-27622 に対しても， AC_{pre} および AC_{post} と AC_{exp} との比較を行った．

表 3，表 4 に，5.3.1 節と同様に，アクセス制御を抽出した結果とパッチに関連するアクセス制御の結果を示す．

$(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Deny, Deny) となる正しく脆弱性修正されていたアクセス制御は平均 7 件であった．これらも 5.3.1 節と同様にセキュリティパッチによって正しく修正された脆弱性が反映された結果であった．

しかし, $(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Allow, Deny) となるアクセス制御の中に，セキュリティパッチによって正しく修正されたにも関わらず，AC_{exp} の推定が不十分であったため，見逃されていたものが 1 件あった．この見逃しも含めて，$(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Allow, Deny) となったアクセス制御が平均 100.3 件存在した．これは，CVE の説明文で指定されていた具体的なファイル名が config.xml のみであり，CVE に対するセキュリティパッチがどのファイルを修正して，どのような機密操作を防御するかを推測することが難しいためだと考えられる．

さらに, $(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Deny, Allow) となるアクセス制御が 0 件であったことから，CVE-2025-31721 の修正によって追加された ComputerSet.java に対する PC を推定することはできていなかった．これも，CVE の説明文が具体的な修正ファイルを指していないことが原因と考えられる．

以上より，RQ2 の回答は以下となる．

RQ2 に対する回答： 実験結果からは，不完全なセキュリティパッチに残存するアクセス制御脆弱性の修正漏れの評価に提案手法は有効ではなかった．AC_{exp} を生成する際に利用する CVE の説明文の内容が具体的なファイルを示していない場合，本手法では修正漏れを評価することは難しい．

6.　考察

6.1　AC_{exp} の生成

本研究では，セキュリティパッチによって期待されるアクセス制御生成のために該当する CVE の説明文を入力の一つとして利用した．CVE の説明文は脆弱な箇所を具体的に表すものや，そうでないものがあり，本手法ではこの説明文の特性によって，AC_{exp} の精度が変わると考えられる．

今回実験対象とした CVE を比較すると, CVE-2020-10194 は説明文に脆弱性がある具体的なファイルパス名まで記述されていたが, CVE-2025-27622 ではそうではなかった．実験結果でも，CVE-2025-27622 は $(AC_{pre}, AC_{exp}, AC_{post})$ = (Allow, Allow, Deny) となるようなアクセス制御が多く存在し，さらにそれが異常な修正ではなかった．

6.2　本手法の限界

本手法には主に，アクセス制御条件の意味的比較，PC の妥当性判定，およびアクセス制御の表現力に関して三つの限界が存在する．

アクセス制御条件の意味的比較: 本手法では，アクセス制御に用いられる条件式そのものの意味的変化を自動的に評価することができない．本手法では，エントリから機密操作までの実行経路上に PC が存在するか，またその配置がパッチ適用前後でどのように変化したかを解析することができる．一方で，既存の PC に含まれる条件式が変更された場合，その変更によってアクセス可能な主体や実行条件の集合が拡大したのか，あるいは縮小したのかを自動的に判断することはできない．例えば，ある条件式に対して論理項が追加・削除された場合や，複数の条件式の組合せが変更された場合には，構文上の差分を検出することは可能であっても，それがアクセス制御を強化する変更なのか，緩和する変更なのかを判定するためには条件式の意味を解釈する必要がある．本実験では，このような PC 内部の条件変更については変更箇所を抽出した上で，人間がその内容を確認する必要があった．

PC の妥当性判定: エントリから機密操作までの経路上

© 2026 Information Processing Society of Japan

に PC が存在することのみをもって，アクセス制御が適切であると保証することはできない．これは，本研究のアクセス制御生成手法が PC の存在や配置を扱う一方で，その PC が表す条件を十分に評価していないためである．例えば，本来は管理者のみが実行可能であるべき処理に対して，一般ユーザを含む広い条件を許可する PC が存在していた場合，経路上には PC が存在するものの，アクセス制御脆弱性は依然として残っている可能性がある．したがって，本手法によって確認できるのは，主として「どの経路に認可処理が存在するか」という構造的な性質であり，「その認可処理が適切なアクセス境界を実現しているか」という意味的な性質については別途確認が必要である．

アクセス制御の表現力: 第三に，本手法で構築するアクセス制御表現は，プログラム中のエントリ，機密操作，および PC の関係を中心としたものであり，一般的なアクセス制御ポリシーが持つ主体や資源に関する詳細な属性を十分には表現していない．すなわち，Role-Based Access Control におけるユーザの役割や，Atribute-Based Access Control における主体・資源・環境の属性，組織階層，所有関係などを明示的にモデル化していない．そのため，アクセス制御条件が主体や資源の属性に依存する脆弱性については，本手法のモデルだけでは十分に表現できない可能性がある．

7. おわりに

本研究では，アクセス制御脆弱性を対象として，セキュリティパッチによるアクセス制御の変化に着目したパッチ評価手法を提案した．提案手法では，パッチ適用前のプログラムから抽出したアクセス制御，脆弱性情報やパッチ適用前のプログラムを基に LLM によって構築したパッチ適用後に期待されるアクセス制御，およびパッチ適用後のプログラムから抽出したアクセス制御の三者を比較する．これにより，脆弱性の原因となるアクセスが適切に制限されたかを確認するだけでなく，本来維持されるべきアクセスの過剰な制限や，修正されるべきアクセスの残存といった，パッチによるアクセス制御上の影響を評価することを目的とした．

結果として，提案手法では十分にアクセス制御の変化を評価することは難しかった．これは，パッチによって期待されるアクセス制御生成に限界があることが主な原因と考えられる．また，対応可能なアクセス制御脆弱性も少ないため，今後は対応可能な脆弱性を増やすことも課題となる．

参考文献

[1] FIRST Forecasting team, “The 2026 vulnerability forecast update: Navigating the ai epoch.” https://www.first.org/blog/20260615-vulnerability-forecast-update, visited on 2026-8-20.

[2] Cloud Security Alliance, “Ai vulnerability storm.” https://labs.cloudsecurityalliance.org/mythos-ciso/, visited on 2026-8-20.

[3] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz, “Verification and change-impact analysis of access-control policies,” in *Proceedings of the 27th International Conference on Software Engineering*, pp. 196–205, Association for Computing Machinery, 2005.

[4] Y. Shi, F. Liu, G. Yang, Y. Zhang, Y. Cao, E. Li, X. Tan, X. Luo, M. Yang, and S. Chen, “Facilitating access control vulnerability detection in modern java web applications with accurate permission check identification,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 10683–10698, 2025.

[5] Y. Wu, Z. Lin, Y. Chen, D. K. Le, D. Mu, and X. Xing, “Mitigating security risks in linux with KLAUS: A method for evaluating patch correctness,” in *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 4247–4264, USENIX Association, 2023.

[6] S. Woo, E. Choi, and H. Lee, “A large-scale analysis of the effectiveness of publicly reported security patches,” *Computers & Security*, vol. 148, p. 104181, 2025.

[7] T. Le-Cong, B. Le, T. Murray, M. Pradel, and C. Cadar, “PatchGuru: Patch oracle inference from natural language artifacts with large language models,” 2026.

[8] F. Sun, L. Xu, and Z. Su, “Static detection of access control vulnerabilities in web applications,” in *20th USENIX Security Symposium (USENIX Security 11)*, USENIX Association, 2011.

[9] T. Berger, R. W. Nguempnang, K. Sohr, and R. Koschke, “Static extraction of enforced authorization policies—SeeAuthz,” in *2020 IEEE 20th International Working Conference on Source Code Analysis and Manipulation*, pp. 187–197, IEEE, 2020.

[10] T. Zhang, W. Shen, D. Lee, C. Jung, A. M. Azab, and R. Wang, “PeX: A permission check analysis framework for linux kernel,” in *28th USENIX Security Symposium (USENIX Security 19)*, pp. 1205–1220, USENIX Association, 2019.

[11] W. Zhang, D. Bali, J. Kerney, A. Panda, and S. Shenker, “Extracting database access-control policies from web applications,” in *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, USENIX Association, 2026.

[12] C. Xiang, Y. Wu, B. Shen, M. Shen, H. Huang, T. Xu, Y. Zhou, C. Moore, X. Jin, and T. Sheng, “Towards continuous access control validation and forensics,” in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 113–129, ACM, 2019.

[13] C. Xiang, L. Zhong, E. Mugnier, N. Nguyen, Y. Zhou, and T. Xu, “Testing access-control configuration changes for web applications,” *arXiv preprint arXiv:2505.12770*, 2025.

[14] “Codeql.” https://codeql.github.com/, visited on 2026-8-20.

[15] “cve-2020-10194.” https://nvd.nist.gov/vuln/detail/CVE-2020-10194, visited on 2026-8-20.

[16] “cve-2025-27622.” https://nvd.nist.gov/vuln/detail/CVE-2026-27622, visited on 2026-8-20.

[17] “cve-2025-31721.” https://nvd.nist.gov/vuln/detail/CVE-2026-31721, visited on 2026-8-20.

[18] “Github advisory cve-2025-31721.” https://github.com/advisories/GHSA-wr6w-jxg7-qpfh, visited on 2026-8-21.

© 2026 Information Processing Society of Japan